Integrating the A2A Protocol with Amazon SQS

A practical guide to agent-to-agent communication over queues | November 4, 2026

A2A has no official SQS binding, so you have to build the SQS layer yourself. The spec's standard bindings are JSON-RPC 2.0 over HTTP/SSE, gRPC, and HTTP/REST. There are two reasonable ways to bring SQS in: a hybrid design that keeps an A2A HTTP endpoint and uses SQS behind it, or a custom A2A protocol binding that carries A2A messages directly over queues.

Option 1: Hybrid (SQS behind an A2A HTTP endpoint)

This is the easiest option and stays fully spec-compliant. Agent B exposes a normal A2A endpoint and uses SQS internally to decouple the request from the work. One AWS write-up of this pattern returns a task with **submitted** status immediately, queues the message to SQS for background processing, and has the client poll `tasks/get` until the agent finishes.

Flow

- Agent A calls `SendMessage` on B's HTTP endpoint.
- B's handler creates the Task in a store (DynamoDB or Postgres), enqueues the work to SQS, and returns the Task as **submitted**.
- A worker consumes the queue, runs your `AgentExecutor`, and updates the task state.
- Agent A learns the result by polling `GetTask`, by streaming, or by A2A push notifications. Push notifications let the caller register a webhook, and the result is delivered there when the work is done.

Use this when agent A can reach B over HTTP and you only want SQS for buffering, retries and scaling.

Option 2: A custom A2A binding over SQS

Use this when the agents can only talk through queues (private networks, no ingress, strict decoupling). The spec allows this: custom bindings are declared in the Agent Card's `supportedInterfaces` list, with a transport URI, an endpoint URL and the A2A version. They must also behave consistently with the standard bindings, map every A2A error type to the transport's native errors, and define how authentication works.

Topology

- One request queue per agent, e.g. `agent-b-requests.fifo`.
- One reply queue per client agent, e.g. `agent-a-replies.fifo`.
- The reply queue URL travels in a message attribute, so B knows where to answer.

Envelope

The SQS body is the normal A2A JSON-RPC payload, and the metadata goes in message attributes. The body below is illustrative. Method names and field shapes depend on the spec version, so check the normative proto for the version you target.

```
{
  "jsonrpc": "2.0",
  "id": "req-7f3a",
  "method": "SendMessage",
  "params": { "message": { "messageId": "m-1", "role": "ROLE_USER",
                           "parts": [{ "text": "Summarise Q3 sales" }] } }
}
```

Attributes: `a2a-version`, `reply-to` (queue URL), `correlation-id` (the JSON-RPC id), and `a2a-task-id` once one exists.

Agent A (sender)

```
import json, uuid, boto3
sqs = boto3.client("sqs", region_name="ap-south-1")

REQ_Q   = "https://sqs.ap-south-1.amazonaws.com/<acct>/agent-b-requests.fifo"
REPLY_Q = "https://sqs.ap-south-1.amazonaws.com/<acct>/agent-a-replies.fifo"

def send(text: str, context_id: str | None = None) -> str:
    req_id = str(uuid.uuid4())
    body = {"jsonrpc": "2.0", "id": req_id, "method": "SendMessage",
            "params": {"message": {"messageId": req_id, "role": "ROLE_USER",
                                   "parts": [{"text": text}]}}}
    sqs.send_message(
        QueueUrl=REQ_Q,
        MessageBody=json.dumps(body),
        MessageGroupId=context_id or req_id,      # ordering per conversation
        MessageDeduplicationId=req_id,
        MessageAttributes={
            "a2a-version": {"DataType": "String", "StringValue": "1.0"},
            "reply-to":    {"DataType": "String", "StringValue": REPLY_Q},
            "correlation-id": {"DataType": "String", "StringValue": req_id},
        },
    )
    return req_id
```

Agent B (consumer)

A worker loop feeds each request into your A2A handler and sends every resulting event back on the reply queue.

```
def serve(handle):   # handle(req) yields Task / status-update / artifact-update dicts
    while True:
        resp = sqs.receive_message(QueueUrl=REQ_Q, MaxNumberOfMessages=1,
                                   WaitTimeSeconds=20, MessageAttributeNames=["All"])
        for m in resp.get("Messages", []):
            a = m["MessageAttributes"]
            reply_q = a["reply-to"]["StringValue"]
            corr    = a["correlation-id"]["StringValue"]
            req     = json.loads(m["Body"])
            try:
                for seq, event in enumerate(handle(req)):
                    out = {"jsonrpc": "2.0", "id": req["id"], "result": event}
                    sqs.send_message(QueueUrl=reply_q, MessageBody=json.dumps(out),
                                     MessageGroupId=corr,
                                     MessageDeduplicationId=f"{corr}-{seq}")
            except Exception as e:
                err = {"jsonrpc": "2.0", "id": req["id"],
                       "error": {"code": -32603, "message": str(e)}}
                sqs.send_message(QueueUrl=reply_q, MessageBody=json.dumps(err),
                                 MessageGroupId=corr,
                                 MessageDeduplicationId=f"{corr}-err")
            sqs.delete_message(QueueUrl=REQ_Q, ReceiptHandle=m["ReceiptHandle"])
```

Wire `handle` to the Python SDK's request handler, so your existing `AgentExecutor` logic is reused and you only write the transport. The SDK implements spec 1.0 with a compatibility mode for 0.3, and the 1.0 release was a breaking change with significant API changes, so check the 1.x class names rather than

copying older examples.

Agent Card

Advertise the interface in the card:

```
"supportedInterfaces": [{
  "url": "https://sqs.ap-south-1.amazonaws.com/<acct>/agent-b-requests.fifo",
  "protocolBinding": "https://yourcompany.com/bindings/a2a-sqs/v1",
  "protocolVersion": "1.0"
}]
```

Standard A2A clients will not understand this binding, so both agents need your SQS client code. Keep the card itself reachable (HTTP or S3), or exchange it by config.

Design points

- **Streaming:** SQS has no streaming. Send each status or artifact update as its own message on the reply queue, ordered by FIFO `MessageGroupId` = task or correlation ID.
- **Correlation:** map the JSON-RPC `id` and the A2A `taskId`/`contextId` to message attributes so replies can be matched and multi-turn conversations stay in one group.
- **Long-running tasks:** do not hold a message in flight for minutes. Persist the task first, then delete the message. Otherwise extend visibility with `ChangeMessageVisibility` heartbeats.
- **Duplicates and failures:** standard queues deliver at least once. AWS's agentic-AI guidance recommends dead letter queues on critical queues, queue-depth monitoring, and idempotency keys against duplicate delivery.
- **Large payloads:** SQS caps message size. Put big artifacts in S3 and pass a URL in the A2A Part.
- **Security:** once a message is in SQS, no transport protects it, and tampering between delivery and consumption is invisible. AWS recommends server-side encryption with customer-managed KMS keys, scoped per trust zone, plus verifying authenticity and integrity when consuming. In practice that means per-queue IAM policies and signing the envelope (HMAC or KMS sign) so B can verify who sent it.
- **Errors:** your binding spec must map A2A errors (e.g. `TaskNotFoundError`) to the error format you use on the reply queue. This is a requirement for custom bindings.

If both agents run on AWS and do not need queue semantics, AgentCore Runtime already supports A2A natively over HTTP with session isolation and built-in authentication, which avoids building any of this.

Sources

- A2A custom protocol bindings: https://a2a-protocol.org/latest/topics/custom-protocol-bindings
- A2A quick reference (v1.0.0): https://a2a-protocol.org/llms-reference.txt
- Building a scalable A2A protocol on AWS (DEV): https://dev.to/suraj_khaitan_f893c243958/building-a-scalable-agent-to-agent-a2a-communication-protocol-on-aws-1g0i
- AWS Agentic AI Lens: AGENTPERF04: https://docs.aws.amazon.com/wellarchitected/latest/agentic-ai-lens/agentperf04.html
- AWS Agentic AI Lens: AGENTSEC06-BP01: https://docs.aws.amazon.com/wellarchitected/latest/agentic-ai-lens/agentsec06-bp01.md
- A2A Python SDK: https://github.com/a2aproject/a2a-python
- A2A Protocol Explained (push notifications): https://eco.com/support/en/articles/14845481-a2a-agent-to-agent-protocol-explained